Assignment for the position of Scientific Assistant in Industrial Robotics and Manufacturing

Abdalsalam Housin · 8 November 2026

1 The task

In this task, two xArm6 robots build the frame of drawing AMZ-SA-26-001 from four steel U-profiles: two Pos 1 (U40×20×2, 320 mm) on the base and two Pos 2 (U30×15×2, 280 mm) across them. As a reality check, I explain how I would meet the tolerances when the robots are not precise enough.

2 The requirements

Table 1: The eight requirements of the drawing, each with a tolerance of 0.05 mm.

1	Pos 1 to Pos 1, centre lines	120 mm	5	Pos 1 to Pos 1	parallel
2	Pos 2 to Pos 2, centre lines	120 mm	6	Pos 2 to Pos 1	perpendicular
3	end of Pos 1 to Pos 2 centre line (4 places)	100 mm	7	top of the frame to the base	parallel
4	overhang of Pos 2 (4 places)	60 mm	8	faces where the profiles touch	no gap

3 The problems

Problem 1: the robots are not precise enough. The xArm6 repeats a taught point within ±0.1 mm [1]. That is two times the tolerance in Table 1, and gripping and letting go add more error.

Problem 2: the parts are not precise enough. ISO 2768-m [2] allows ±0.5 mm on lengths and ±0.2 mm on heights. A Pos 1 that is 0.4 mm too long cannot meet both 100 mm distances.

4 The solution

The robots only bring the profiles near their place, and my fixture pushes them into place.

4.1 Fixture

All distances on the drawing are symmetric to the middle of the frame, so my fixture centres each profile and has no fixed stops. Fig. 1 shows the CAD model (`fixture_with_profiles.step`).

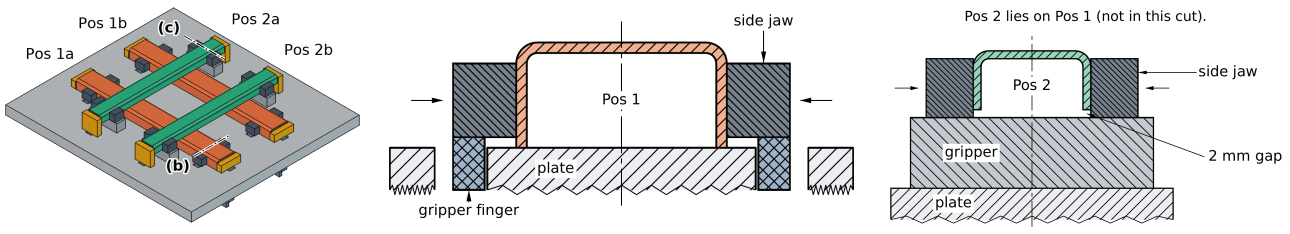
Each profile has two pneumatic parallel grippers that close side jaws on its legs (Fig. 1b, c). Two end jaws on slides under the plate push on its ends. One air cylinder moves both end jaws. Their racks turn one pinion, so each end gets half of any length error.

Pneumatic valves open and close all jaws. Each jaw opens 3 mm. That leaves room for a part that lands up to 2 mm off its place. When the jaws close, they push the part into its place. Pos 2 lies on both Pos 1, and its own weight holds it down.

4.2 Robots and sequence

The two robots stand opposite each other, 480 mm from the fixture centre. Each robot has a rack with one Pos 1 and one Pos 2. Robot 1 places Pos 1a and Pos 2a, robot 2 places Pos 1b and Pos 2b.

For each part, the robot picks it at the rack, sets it down in the fixture and opens the gripper. Then the end jaws close, and after them the side jaws. Only one robot may work at the fixture at a time. Pos 2 comes after both Pos 1. I would teach the place positions by hand on the real fixture.



(a) CAD model with the profiles **(b)** Section through Pos 1 **(c)** Section through Pos 2

Figure 1: The fixture: Pos 1 orange, Pos 2 green, side jaws dark grey, end jaws yellow.

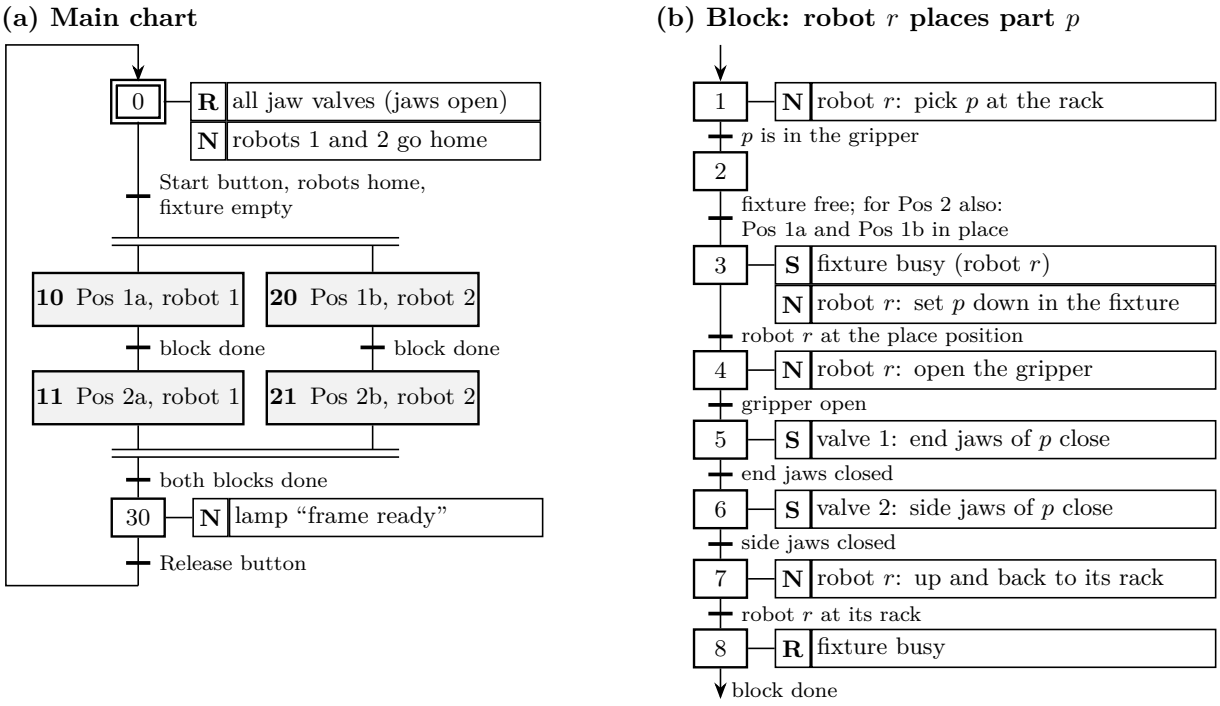


Figure 2: The PLC program as a sequential function chart (SFC).

4.3 Control with a PLC

I would run the cell with a PLC. Fig. 2 shows my program in sequential function chart (SFC). SFC is a programming language of IEC 61131-3 [3] to define a sequence and a state machine. Each step is a state. The PLC goes to the next step when the condition under it is true. Chart (a) starts both robots together. Each part is one block of chart (b), where the PLC tells the robot what to do and switches the jaw valves. An N action is on while its step is active, S switches it on and R switches it off. The signal "fixture busy" keeps the other robot away.

5 Simulation

I tested the cell and the sequence in MuJoCo [4] with the xArm6 models of UFACTORY [5] (Fig. 3, `simulation.py`). All four parts are in place after 23.6 s, and the robots never come closer than 259 mm.

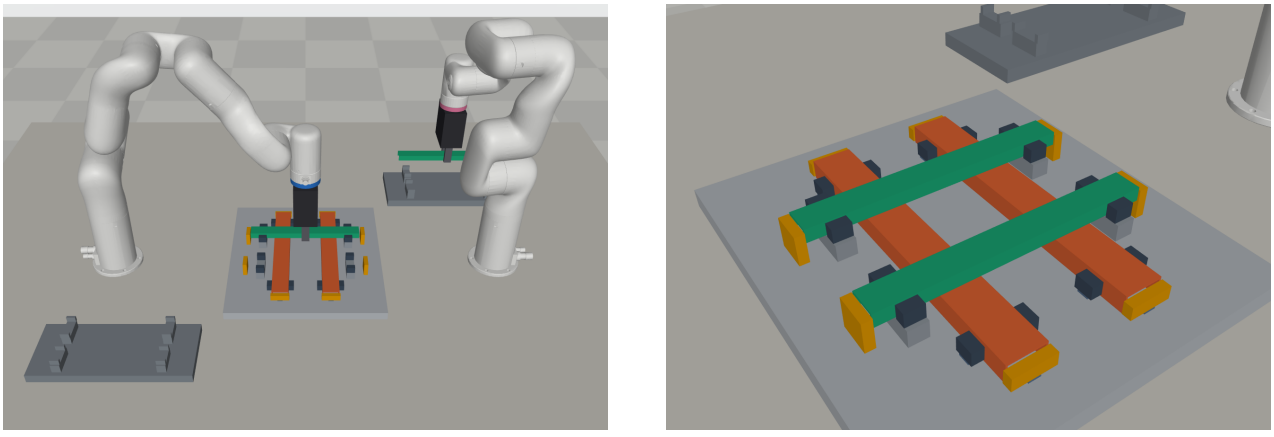


Figure 3: The MuJoCo simulation: robot 1 sets Pos 2a down (left), all four parts in place (right).

References

[1] UFACTORY, xArm 6 technical specifications, `https://docs.xarm.ufactory.cc`.
[2] ISO 2768-1:1989, General tolerances, Part 1: Linear and angular dimensions.
[3] IEC 61131-3:2013, Programmable controllers, Part 3: Programming languages.
[4] E. Todorov, T. Erez, Y. Tassa, MuJoCo: a physics engine for model-based control, IROS 2012.
[5] UFACTORY, xarm_ros2 robot description (BSD-3-Clause), `https://github.com/xArm-Developer/xarm_ros2`.

Disclosure: I used agentic AI coding to help me develop the simulation faster.